EXPERIMENT NO. 11

EM Algorithm for Two Coins

1. Problem + Algorithm

Experiment No.: Class Question
Title: EM Algorithm for Two Coins
Aim: Implement the Expectation-Maximization (EM) algorithm to estimate the head probabilities of two coins when the coin used in each round is hidden.
Problem Statement: Five rounds of 10 coin tosses are recorded, and each round used either coin A or coin B, but which coin was used is unknown. Estimate the probability of heads of each coin by alternating between estimating which coin produced each round and re-estimating the head probabilities.

Algorithm

- Count the heads and tails in each round.
- Initialize θ_A, θ_B and the mixing weights π_A, π_B.
- E-step: compute the probability that each round came from coin A or coin B.
- M-step: update each θ from the expected heads and expected tosses of that coin.
- Repeat until the estimates stop changing.
- Return the estimated head probabilities.

2. Program

Key parameter settings

```
sequences = [
    "HHHTHHHTHH",
    "TTHTTTHTTT",
    "HHHHTHHHHT",
    "TTTHTTTTHT",
    "HTHHHTHHTH"
]

theta_A = 0.6
theta_B = 0.4

pi_A = 0.5
pi_B = 0.5

max_iterations = 1000
tolerance = 1e-10
```

Function definitions

```
data = []

for sequence in sequences:
    heads = sequence.count("H")
    tails = sequence.count("T")
    data.append([heads, tails])

df = pd.DataFrame(data, columns=["Heads", "Tails"], index=[f"Round {i}" for i in range(1, 6)])
```

Core algorithm

```
history = []
for iteration in range(max_iterations):
    old_A = theta_A
    old_B = theta_B
    heads = df["Heads"].values
    tails = df["Tails"].values
    likelihood_A = pi_A * (theta_A ** heads) * ((1 - theta_A) ** tails)
    likelihood_B = pi_B * (theta_B ** heads) * ((1 - theta_B) ** tails)
    responsibility_A = likelihood_A / (likelihood_A + likelihood_B)
    responsibility_B = 1 - responsibility_A
    expected_heads_A = np.sum(responsibility_A * heads)
    expected_tosses_A = np.sum(responsibility_A * (heads + tails))
    expected_heads_B = np.sum(responsibility_B * heads)
    expected_tosses_B = np.sum(responsibility_B * (heads + tails))
    theta_A = expected_heads_A / expected_tosses_A
    theta_B = expected_heads_B / expected_tosses_B
    history.append({
        "Iteration": iteration + 1,
        "Theta A": theta_A,
        "Theta B": theta_B,
        "Expected Heads A": expected_heads_A,
        "Expected Tosses A": expected_tosses_A,
        "Expected Heads B": expected_heads_B,
        "Expected Tosses B": expected_tosses_B
    })
    if abs(theta_A - old_A) < tolerance and abs(theta_B - old_B) < tolerance:
        break
results = pd.DataFrame(history)
```

The complete executable program can be maintained digitally through Github.

3. Output + Analysis

Objective Function: Maximize the log-likelihood of the observed head and tail counts under a mixture of two coins
Search Interval: $0 \le \theta_A \le 1, 0 \le \theta_B \le 1$ (5 rounds of 10 tosses each)

Settings

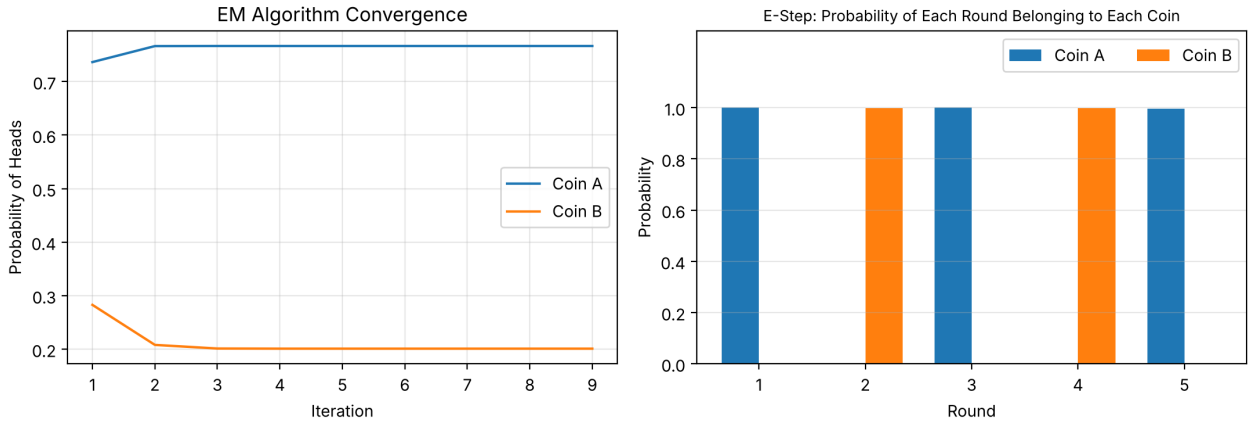
- Initial Solution: $\theta_A = 0.60, \theta_B = 0.40$
- Mixing Weights: $\pi_A = \pi_B = 0.50$
- Stopping Tolerance: 1e-10 (converged in 9 iterations)
- Maximum Iterations: 1000

Program Execution

```
Coin A Head Probability : 0.766444
Coin A Tail Probability : 0.233556
Coin B Head Probability : 0.201007
Coin B Tail Probability : 0.798993
Total Iterations        : 9
```

Execution Status: Successfully Executed

Convergence Graph



X-axis → Left: Iteration; Right: Round
Y-axis → Left: Probability of heads; Right: Probability of belonging to each coin

Analysis

EM converges in 9 iterations to $\theta_A = 0.7664$ and $\theta_B = 0.2010$. Rounds 1, 3 and 5 (8, 8 and 7 heads) are assigned to coin A with probability above 0.996, and rounds 2 and 4 (2 heads each) to coin B with probability 0.999, so the two coins are separated almost completely.

Parameter Modification

S. No.	Parameter	Original	Modified	Original Result	Modified Result	Observation
1	Initial θ_A (θ_B = 0.4)	0.6	0.4	θ_A = 0.7664, θ_B = 0.2010 (9 iterations)	$\theta_A = \theta_B$ = 0.5400 (2 iterations)	Equal starting values give every round equal responsibility, so the coins never separate; both settle at the overall head rate 0.54.
2	Initial θ_B (θ_A = 0.6)	0.4	0.1	θ_A = 0.7664, θ_B = 0.2010 (9 iterations)	θ_A = 0.7664, θ_B = 0.2010 (8 iterations)	A well-separated start reached the same estimates one iteration sooner.
3	Mixing weight π_A	0.5	0.8	θ_A = 0.7664, θ_B = 0.2010 (9 iterations)	θ_A = 0.7655, θ_B = 0.2002 (9 iterations)	Favouring coin A moved the estimates only slightly; the weights are fixed in this program, not re-estimated.
4	Stopping Tolerance	1e-10	1e-3	θ_A = 0.7664, θ_B = 0.2010 (9 iterations)	θ_A = 0.7664, θ_B = 0.2010 (4 iterations)	A looser tolerance stopped after 4 iterations with essentially the same estimates.

EXPERIMENT NO. 12

EM Algorithm for Gaussian Mixture Model

1. Problem + Algorithm

Experiment No.: 15
Title: EM Algorithm for Gaussian Mixture Model
Aim: Implement the EM algorithm from scratch to fit a Gaussian Mixture Model (GMM) to the Palmer Penguins data.
Problem Statement: Fit a 3-component GMM with full covariance matrices to four standardized body measurements of 342 penguins using EM, and compare the recovered clusters with the true species labels.

Algorithm

- Load the data, drop missing rows and standardize the four features.
- Initialize means from random observations, covariances from the data and equal weights.
- E-step: compute the responsibility of each component for each penguin.
- M-step: update the weights, means and covariances.
- Track the log-likelihood and repeat until it stops improving.
- Run several random restarts and keep the best solution.
- Assign each penguin to its most probable component and compare with the species.

2. Program

Key parameter settings

```
df = pd.read_csv("penguins_size.csv")
features = ["culmen_length_mm", "culmen_depth_mm", "flipper_length_mm", "body_mass_g"]
df = df.dropna(subset=features).reset_index(drop=True)
X_raw = df[features].values
mean_ = X_raw.mean(axis=0)
std_ = X_raw.std(axis=0)
X = (X_raw - mean_) / std_
y = df["species"].values
n, d = X.shape

K = 3
```

Function definitions

```
def log_gaussian(X, mu, Sigma):
    d = X.shape[1]
    diff = X - mu
    sign, logdet = np.linalg.slogdet(Sigma)
    sol = np.linalg.solve(Sigma, diff.T).T
    maha = np.sum(diff * sol, axis=1)
    return -0.5 * (d * np.log(2 * np.pi) + logdet + maha)

def logsumexp(a, axis=1):
    m = a.max(axis=axis, keepdims=True)
    return (m + np.log(np.exp(a - m).sum(axis=axis, keepdims=True))).squeeze(axis)

def e_step(X, pi, mu, Sigma):
    K = len(pi)
    log_p = np.column_stack([np.log(pi[k]) + log_gaussian(X, mu[k], Sigma[k]) for k in range(K)])
    log_norm = logsumexp(log_p)
    resp = np.exp(log_p - log_norm[:, None])
    return resp, log_norm.sum()

def m_step(X, resp, reg=1e-6):
    n, d = X.shape
    Nk = resp.sum(axis=0)
    pi = Nk / n
    mu = (resp.T @ X) / Nk[:, None]
    Sigma = np.empty((len(Nk), d, d))
    for k in range(len(Nk)):
        diff = X - mu[k]
        Sigma[k] = (resp[:, k, None] * diff).T @ diff / Nk[k] + reg * np.eye(d)
    return pi, mu, Sigma
```

Core algorithm

```
def fit_em(X, K, seed=0, max_iter=500, tol=1e-8):
    rng = np.random.default_rng(seed)
    n, d = X.shape
    mu = X[rng.choice(n, K, replace=False)]
    Sigma = np.array([np.cov(X.T) for _ in range(K)])
    pi = np.full(K, 1 / K)
    ll_history = []
    for _ in range(max_iter):
        resp, ll = e_step(X, pi, mu, Sigma)
        ll_history.append(ll)
        pi, mu, Sigma = m_step(X, resp)
        if len(ll_history) > 1 and abs(ll_history[-1] - ll_history[-2]) < tol:
            break
    resp, ll = e_step(X, pi, mu, Sigma)
    ll_history.append(ll)
    return {"pi": pi, "mu": mu, "Sigma": Sigma, "resp": resp, "ll": ll, "history": ll_history}

runs = [fit_em(X, K, seed=s) for s in range(10)]
best = max(runs, key=lambda r: r["ll"])
```

The complete executable program can be maintained digitally through Github.

3. Output + Analysis

Objective Function: Maximize the log-likelihood $\Sigma \log \Sigma_k \pi_k N(x \mid \mu_k, \Sigma_k)$ of the data under the mixture
Search Interval: 342 penguins, 4 standardized features (culmen length, culmen depth, flipper length, body mass)

Settings

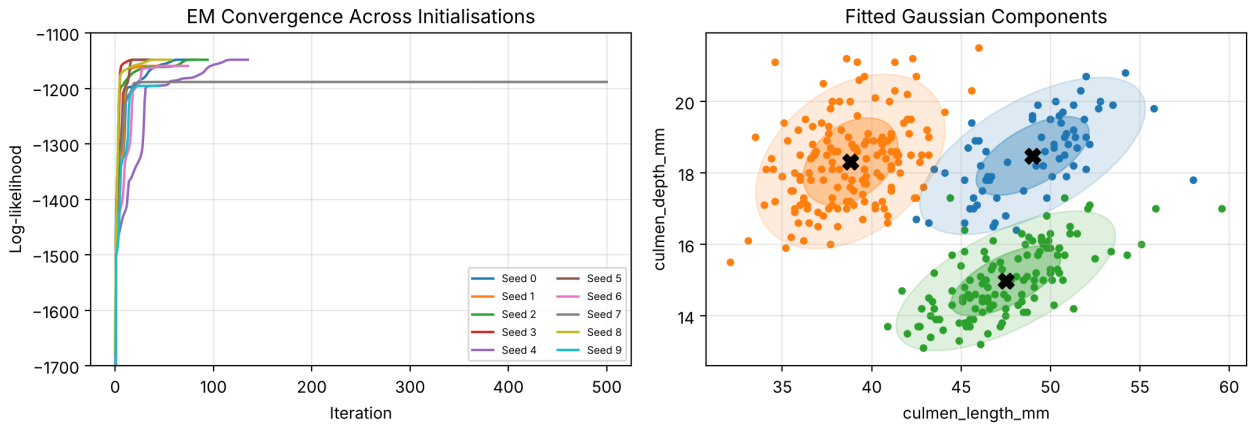
- Initial Solution: means at 3 random observations, data covariance, weights 1/3
- Number of Components: K = 3 (full covariance)
- Random Restarts: 10 seeds, best log-likelihood kept
- Stopping Tolerance: 1e-8 (maximum 500 iterations)
- Covariance Regularisation: 1e-6

Program Execution

```
Observations           : 342 (4 standardized features)
Best Log-likelihood    : -1148.4374
sklearn GaussianMixture: -1148.4424
ARI vs species         : 0.9603
Best K by BIC          : 3
Best K by AIC          : 6
```

Execution Status: Successfully Executed

Convergence Graph



X-axis → Left: Iteration; Right: Culmen length (mm)
Y-axis → Left: Log-likelihood; Right: Culmen depth (mm)

Analysis

The best of 10 random starts reaches a log-likelihood of −1148.44, the same as sklearn, and assigns 337 of 342 penguins to the right species (ARI 0.960): all 123 Gentoo are recovered, 2 Adelie fall in the Chinstrap component and 3 Chinstrap in the Adelie component. Six of the ten starts reach this optimum and four stall at lower values, and BIC is lowest at K = 3.

Parameter Modification

S. No.	Parameter	Original	Modified	Original Result	Modified Result	Observation
1	Number of components K	3	2	Log-likelihood = −1148.437, ARI = 0.960, BIC = 2553.61	Log-likelihood = −1208.795, ARI = 0.655, BIC = 2586.80	Two components cannot separate three species; the likelihood falls and BIC rises.
2	Number of restarts	10	1 (seed 7)	Log-likelihood = −1148.437, ARI = 0.960	Log-likelihood = −1188.486, ARI = 0.520	A single run stalled in a local optimum; 4 of the 10 seeds did, so restarts are needed.
3	Features used	4	2 (culmen only)	Log-likelihood = −1148.437, ARI = 0.960	Log-likelihood = −821.969, ARI = 0.912	Fewer features lose information (ARI falls); log-likelihoods are not comparable across different dimensions.
4	Covariance regularisation	1e-6	1e-1	Log-likelihood = −1148.437, ARI = 0.960	Log-likelihood = −1239.108, ARI = 0.945	A large added variance inflates every covariance and lowers the likelihood.